TRABAJO PRÁCTICO · UNIDAD 7: NOSQL Y NUEVAS FORMAS DE ALMACENAMIENTO

Modelado Documental, Migración y Verificación Cruzada: PostgreSQL vs. MongoDB en Food Store

Materia: Bases de Datos II **Proyecto:** Food Store **Entorno:** PostgreSQL 18 + MongoDB 7.0/8.0 (mongosh) **Fecha:** Noviembre 2026
Integrantes del Equipo: Ivana Gutierrez · Rayssa Alejo · Julieta Chavez · Brisa Muñoz
Repositorio GitHub: https://github.com/rayssa-ale/food-store **Rama / Entrega:** `tp7-ivana` (o `main`)

1. Introducción y Alcance del Trabajo

El presente trabajo implementa la transición de un subconjunto transaccional crítico del sistema **Food Store** desde un esquema relacional normalizado (PostgreSQL) hacia una base de datos orientada a documentos (**MongoDB**). Se evalúa el impacto de la desnormalización mediante **Embedding** de cabeceras, líneas de detalle y clientes, garantizando la paridad estricta de consultas y contrastando las recomendaciones generadas por herramientas de Inteligencia Artificial frente a la documentación técnica oficial de ambos paradigmas.

2. Parte 1 · Modelo Documental y Migración (Consignas 1 a 3)

2.1 Mapeo Relacional a Documental (Consigna 1)

En PostgreSQL, el circuito de ventas está articulado por cuatro tablas normalizadas con claves primarias y foráneas: **`pedido`** (PK `id`, FK `usuario_id`), **`detalle_pedido`** (PK `id`, FK `pedido_id`, FK `producto_id`, restricción `UNIQUE(pedido_id, producto_id)`), **`usuario`** (PK `id`) y **`producto`** (PK `id`, FK `categoria_id`).

En MongoDB, para priorizar la **localidad de datos** (*Data Locality*) y responder a la alta frecuencia de lectura del pedido completo en una única operación de entrada/salida, se diseñó la colección **`pedidos`** aplicando incrustación:

```
// Esquema Documental propuesto para la colección 'pedidos' { "_id": ObjectId("673f8a1b2c4e8a1d00000001"), "pedido_id": 1, // Identificador relacional original de PostgreSQL "fecha": ISODate("2026-06-05T20:30:00Z"), "estado": "CONFIRMADO", // PENDIENTE | CONFIRMADO | TERMINADO | CANCELADO "forma_pago": "EFECTIVO", "cliente": { // Objeto embebido con snapshot de datos esenciales "cliente_id": 2, "nombre": "Juan", "apellido": "Pérez", "email": "juan@x.com", "telefono": "2611234567" }, "lineas": [ // Arreglo embebido de ítems (congelamiento de precio histórico) { "producto_id": 1, "nombre": "Muzzarella", "categoria": "Pizzas", "cantidad": 2, "precio_unitario": 3200.00, "subtotal": 6400.00 }, { "producto_id": 7, "nombre": "Coca-Cola 500ml", "categoria": "Bebidas", "cantidad": 3, "precio_unitario": 900.00, "subtotal": 2700.00 } ], "total": 9100.00 // Coherencia matemática sum(subtotales) == total }
```

2.2 Migración de Datos (Consigna 2)

Se elaboró el script `migracion_mongodb.js` ejecutando `insertMany()` con los 10 pedidos del seed representativo de Food Store. Cada documento respeta los identificadores de origen (`pedido_id`, `cliente_id`, `producto_id`), congelando las cantidades, precios de lista históricos e importes consolidados (abarcando estados *CONFIRMADO, TERMINADO* y *CANCELADO*). Se definieron índices B-Tree sobre `pedido_id` (único), `estado`, `total` y un índice multikey sobre `lineas.producto_id`.

2.3 Integridad y Justificación Técnica de Embedding (Consigna 3)

Reglas de Integridad Conservadas Durante la Migración:

- **Regla 1 (Coherencia Matemática y Congelamiento Histórico):** En cada subdocumento de línea se satisface `subtotal == cantidad * precio_unitario` y en la raíz `total == sum(lineas.subtotal)`. El `precio_unitario` es inmutable: captura el valor pactado al facturar, desacoplándose de posteriores modificaciones en el catálogo maestro de precios.
- **Regla 2 (Cardinalidad Mínima y Unicidad de Renglón):** Todo pedido contiene al menos una línea (`minItems: 1`), con `cantidad > 0`. Se preserva la invariante relacional `UNIQUE(pedido_id, producto_id)` agrupando cantidades para evitar líneas duplicadas de un mismo artículo.

Fundamentación de Documentos Embebidos vs. Referencias: Se embeben las *líneas de pedido* y el *cliente esencial* porque constituyen el núcleo atómico del pedido (*Aggregate Root*). El acceso al pedido casi invariablemente requiere conocer qué se consumió y a quién entregarlo. Embeber garantiza lectura de latencia ultra baja en un solo round-trip sin operaciones de `JOIN` ni `$lookup`, y provee **atomicidad transaccional a nivel de documento único** (sin el overhead de coordinadores distribuidos). Por el contrario, **no se embebe el catálogo de productos ni la tabla completa de usuarios**: son entidades maestras con alta tasa de mutación independiente (actualización masiva de precios, stock, contraseñas o roles) que si estuvieran desnormalizadas provocarían anomalías de redundancia y actualización masiva.

3. Parte 2 · Consultas y Verificación Cruzada (Consignas 4 y 5)

3.1 Consultas Ejecutadas en MongoDB (`consultas_mongodb.js`)

- **4.a) Todos los pedidos con identificador y estado:**

```
db.pedidos.find({}, { _id: 1, pedido_id: 1, estado: 1, total: 1 }).sort({ pedido_id: 1 });
```

Resultado: Retorna los 10 pedidos con su ID relacional, ObjectId de MongoDB, estado y total.

- **4.b) Filtro por estado existente ('TERMINADO'):**

```
db.pedidos.find({ estado: "TERMINADO" }, { _id: 0, pedido_id: 1, estado: 1, total: 1, "cliente.nombre": 1 });
```

Resultado: Recupera 7 pedidos (IDs: `2, 3, 4, 6, 7, 9, 10`).

- **4.c) Búsqueda por producto dentro del arreglo embebido (producto_id = 1, 'Muzzarella'):**

```
db.pedidos.find({ "lineas.producto_id": 1 }, { _id: 0, pedido_id: 1, estado: 1, total: 1 });
```

Resultado: Recupera 4 pedidos (IDs: `1, 4, 7, 10`). MongoDB resuelve el filtro de forma nativa mediante su índice multikey sin necesidad de desenrollar el array (`$unwind`).

- **4.d) Filtro por total superior a un umbral justificado ($gt: 8000.00):**

```
db.pedidos.find({ total: { $gt: 8000.00 } }, { _id: 0, pedido_id: 1, total: 1, estado: 1 });
```

Justificación del Umbral ($8.000,00): En el dataset de Food Store, los pedidos presentan una media de $9.880 y una mediana de $8.050 (rango entre $2.800 y $24.100). El umbral de $8.000 discrimina con precisión analítica los pedidos individuales de bajo costo (empanadas/bebidas sueltas) de los pedidos familiares o grupales (múltiples pizzas/combos), aislando los 5 pedidos de mayor contribución a la facturación (IDs: `10, 9, 4, 7, 1`).

3.2 Consultas Equivalentes en PostgreSQL (`consultas_postgresql.sql`)

```
-- Consulta 5.b: Equivalente a 4.b (Filtrar pedidos en estado 'TERMINADO') SELECT p.id AS pedido_id, p.fecha, p.estado, u.nombre || ' ' || u.apellido AS cliente, p.total FROM pedido p JOIN usuario u ON p.usuario_id = u.id WHERE p.estado = 'TERMINADO' ORDER BY p.id ASC; -- Consulta 5.c: Equivalente a 4.c (Pedidos con producto_id = 1 mediante JOIN y DISTINCT) SELECT DISTINCT p.id AS pedido_id, p.fecha, p.estado, p.total FROM pedido p JOIN detalle_pedido dp ON p.id = dp.pedido_id WHERE dp.producto_id = 1 ORDER BY p.id ASC;
```

3.3 Matriz de Verificación y Comparación Cruzada (Consigna 5)

Consulta Evaluada	Cantidad en PostgreSQL	Cantidad en MongoDB	Coinciden los IDs	Detalle de Identificadores Recuperados
Estado seleccionado ('TERMINADO')	**7**	**7**	**SÍ (100%)**	IDs: `[2, 3, 4, 6, 7, 9, 10]`
Producto seleccionado (producto_id = 1)	**4**	**4**	**SÍ (100%)**	IDs: `[1, 4, 7, 10]`

Explicación Técnica de Paridad y Diferencias de Mecanismo: Se comprobó la paridad total en cantidad e identificadores. Sin embargo, los motores operan de manera conceptualmente distinta: En **PostgreSQL**, recuperar los pedidos del producto 1 requiere una operación de reunión relacional (`Hash Join` o `Nested Loop`) entre dos tablas físicas (`pedido` y `detalle_pedido`), demandando la cláusula `DISTINCT` (o un subquery con `EXISTS`) para que la cabecera no se repita si hubiera duplicaciones lógicas. En **MongoDB**, la consulta `db.pedidos.find({ "lineas.producto_id": 1 })` navega directamente sobre el arreglo contenido en el propio documento. Al evaluar la condición multikey, devuelve el documento raíz sin necesidad de uniones externas, deduplicaciones ni consumo extra de memoria para tablas temporales.

4. Parte 3 · Evaluación Tecnológica y Contraste con IA (Consignas 6 y 7)

4.1 Consulta planteada a la Inteligencia Artificial (Consigna 6)

> *«En Food Store disponemos de PostgreSQL con tablas normalizadas (pedido, detalle_pedido, cliente, producto). El patrón dominante es la lectura de pedidos cerrados con sus líneas completas, requiriendo estricta consistencia de precios y stock bajo alto volumen. ¿Conviene migrar pedidos a MongoDB aplicando Embedding?»*

Resumen de la propuesta de la IA: La IA recomendó la migración completa a MongoDB afirmando que: *(1) la eliminación de JOINs hace que las consultas sean siempre más rápidas que en PostgreSQL, y (2) el embedding soluciona la concurrencia y la consistencia sin necesidad de transacciones complejas gracias a la atomicidad del documento.*

4.2 Contraste Crítico contra Documentación Oficial y Evidencia de Laboratorio (Consigna 7)

Afirmación 1 de la IA: *«MongoDB siempre rinde mejor que PostgreSQL para consultar pedidos porque los JOINs relacionales degradan severamente el rendimiento.»*
Contraste Técnico y Documentación Oficial:
La afirmación es una **generalización engañosa**. La documentación oficial de MongoDB (*Data Model Design: Embedded vs. References*) estipula que el embedding beneficia la lectura de documentos puntuales agrupados en una misma página de disco (*Data Locality*). No obstante, para consultas analíticas agregadas (ventas mensuales, ranking de productos, proyecciones parciales), el documento embebido fuerza a transferir metadatos y campos redundantes desde el almacenamiento a la memoria RAM. En contrapartida, las mediciones en PostgreSQL con índices B-Tree (`idx_detalle_pedido_pedido`, `idx_pedido_usuario`) resuelven la reunión relacional en sub-milisegundos (< 1 ms) utilizando `Merge Join` o `Index Only Scan`, manteniendo un *buffer pool* de memoria significativamente más eficiente gracias a la normalización. Por ende, la superioridad de MongoDB no es absoluta, sino estrictamente acotada a lecturas del agregado completo por clave.

Afirmación 2 de la IA: *«En MongoDB no es necesario preocuparse por la integridad del stock o precios históricos porque las operaciones sobre el documento embebido son atómicas.»*
Contraste Técnico y Documentación Oficial:
La IA confunde **atomicidad de documento único** (*single-document atomicity*) con **consistencia transaccional de negocio multi-documento**. La documentación oficial de MongoDB (*Transactions & Atomicity*) advierte que las garantías ACID nativas libres de sobrecosto aplican *únicamente sobre un documento individual*. En Food Store, la creación de un pedido exige: (a) insertar el pedido, (b) decrementar el stock en la colección/tabla `producto`, y (c) validar disponibilidad. En MongoDB, garantizar esto sin condiciones de carrera exige transacciones multi-documento (`session.startTransaction()`), las cuales introducen penalizaciones de latencia y coordinación distribuida similares a las de un motor RDBMS. En contraste, PostgreSQL resuelve esto de forma nativa e inquebrantable mediante restricciones de catálogo (`FOREIGN KEY`, `CHECK (stock >= 0)`) y transacciones ACID nativas, impidiendo inconsistencias que en MongoDB recaerían en validaciones defensivas del código de aplicación.

5. Conclusión Técnica y Dictamen de Arquitectura (Consigna 7)

Dictamen Fundamentado (Máx. 200 palabras · Recuento actual: 172 palabras):
Se desaconseja migrar el núcleo transaccional de Food Store exclusivamente a MongoDB; se recomienda implementar una **arquitectura híbrida (Persistencia Políglota)** sustentada en dos criterios técnicos concluyentes:

1. **Integridad Transaccional y Consistencia Operativa:** La emisión de pedidos demanda consistencia ACID inmediata entre la facturación y el descuento de inventario. PostgreSQL garantiza integridad estructural mediante restricciones nativas en el motor (`CHECK (stock >= 0)`, claves foráneas e índices únicos), previniendo ventas sin stock o datos corruptos sin sobrecosto de coordinación distribuida.
2. **Localidad de Lectura y Desacoplamiento de Carga:** Para la visualización de comandas en cocina, trazabilidad del repartidor y consulta histórica del cliente, el modelo documental con *Embedding* en MongoDB es ampliamente superior, al recuperar el grafo íntegro del pedido en un único acceso a disco sin `JOINs`.

Por tanto, el diseño óptimo conserva a **PostgreSQL como Fuente Única de Verdad (SoT) transaccional**, sincronizando asincrónicamente mediante eventos los pedidos finalizados hacia **MongoDB como almacén de lectura de alto rendimiento**.

6. Referencias Técnicas Consultadas

1. MongoDB Inc. *Data Model Design: Embedded vs. References*. mongodb.com/docs/manual/core/data-model-design/
2. MongoDB Inc. *Query an Array of Embedded Documents*. mongodb.com/docs/manual/tutorial/query-embedded-documents/
3. The PostgreSQL Global Development Group. *PostgreSQL 18 Documentation: Concurrency Control and JSON Support*. postgresql.org/docs/18/
4. Cátedra Bases de Datos II (UTN FRM). *Unidad 7: NoSQL y nuevas formas de almacenamiento.*